Open Access | https://doi.org/10.25080/fpaf9954



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Benchmarking Edge-Accelerated Genomics: A Pilot Study of Unified Memory Architectures in Deep-Sea Metagenomics

Vinay Vyas[1]

[1]Independent Researcher

Abstract

Deep-sea metagenomics involves sequencing total DNA from oceanic samples (water/sediment) to analyze microbial communities without laboratory cultivation. Large-scale deep-sea projects generate terabyte-scale datasets that often exceed the memory and bandwidth capacity of standard GPU clusters. This paper evaluates localized "edge" computing—the practice of processing data at or near the source of data generation to reduce latency and infrastructure overhead—for microbial read binning using the NVIDIA DGX Spark (Grace Blackwell GB10). We implement a GPU-accelerated pipeline utilizing PySpark 3.5, Project Glow, and RAPIDS (cuML), and benchmark it against a CPU-bound scikit-learn baseline using 4-mer frequency matrices derived from the Malaspina deep-ocean expedition dataset (NCBI BioProject PRJNA365132). Across dataset scales from 100k to 2M reads, the GPU pipeline achieves speedups of 10–39 times end-to-end at overlapping 100k–500k scales (95% CI across repeated runs), and extends to 1M–2M reads where the CPU baseline fails, with minimal code changes. GPU acceleration is provided exclusively by cuML ML kernels; Spark data loading ran on CPU at all scales. We assess hardware viability, memory behavior under the 128 GB unified address space, and discuss the capital versus operating expenditure (CAPEX/OPEX) implications of edge computing for resource-constrained marine biology labs.

Keywords GPU acceleration, metagenomics, RAPIDS, unified memory, edge computing, PySpark, deep-sea genomics

1. Introduction

The world's deep ocean remains one of the least studied biomes on Earth. Shotgun metagenomics — sequencing all DNA from an environmental sample — offers a window into this "dark matter" of microbiology, revealing novel organisms, metabolic pathways, and biogeochemical cycles. The Malaspina 2010 circumnavigation expedition collected samples across all major ocean basins, producing one of the largest deep-sea metagenomic datasets publicly available [1].

The computational bottleneck is severe. Binning contigs — grouping assembled sequences by organism of origin — requires high-dimensional dimensionality reduction over matrices with tens of millions of rows. Standard approaches rely on tetranucleotide (4-mer) frequency signatures combined with coverage depth, followed by UMAP and density-based clustering (DBSCAN) [2], [3]. On CPU clusters this process takes hours; on discrete GPU systems it is bottlenecked by PCIe data-transfer overhead when matrices exceed GPU VRAM.

We investigate whether the NVIDIA Grace Blackwell architecture — specifically its 128 GB unified CPU-GPU memory space with no PCIe bus between processor and accelerator — can remove this bottleneck and bring enterprise-grade genomics computing to a single self-contained edge appliance (the DGX Spark) [4]. Our contribution is a transparent, "boots on

Published Jul 13, 2026

Correspondence to
Vinay Vyas
vinaykvyas@gmail.com

Open Access

Copyright © 2026 Vyas. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

the ground" benchmark: same algorithm, same data, same machine — CPU stack versus GPU stack — with honest reporting of setup friction and hardware behavior.

This is directly relevant to the economics of small-lab marine biology research. Deep-sea expeditions are infrequent and expensive; computational resources are often grant-funded and scarce. A one-time capital investment in an edge appliance that matches or exceeds cloud cluster performance offers a compelling alternative to recurring and unpredictable cloud compute costs.

2. Background and Related Work

2.1. Metagenomic Binning

Binning is the process of partitioning assembled or raw metagenomic sequences by their organism of origin. Modern pipelines use k-mer composition (typically k=4, yielding 256 features) as a proxy for genomic signature, since organisms have characteristic nucleotide usage patterns [5]. This produces a high-dimensional feature matrix that is then reduced via PCA and clustered with UMAP + DBSCAN. Representative tools include MetaBAT2 [2] and VAMB [3].

2.2. GPU Acceleration for Bioinformatics

GPU acceleration has been applied to sequence alignment (NVIDIA Parabricks [6]), variant calling, and more recently to ML-based genomic analysis. RAPIDS cuML provides GPU-accelerated implementations of scikit-learn-compatible algorithms including PCA, UMAP, and DBSCAN [7]. The key challenge for genomics is data transfer: moving large matrices from CPU RAM to discrete GPU VRAM via PCIe is a significant overhead, particularly when matrices do not fit in VRAM and must be batched. Existing GB10 developer guides focus on quantized LLM inference workloads [8]; to our knowledge, no prior work benchmarks the platform for metagenomic read binning.

2.3. Unified Memory Architectures

NVIDIA Grace Blackwell (GB10) integrates ARM CPU cores and a Blackwell GPU die on a single package, connected by NVLink-C2C at approximately 600 GB/s aggregate bandwidth [9], [10] — roughly 5 times that of PCIe 5.0 x16. The 128 GB LPDDR5X pool itself delivers $\sim$ 273 GB/s of memory bandwidth [11], which is the effective bottleneck for memory-bound kernels. More critically, the architecture exposes a single coherent address space: any pointer valid on the CPU is valid on the GPU without an explicit copy. This is architecturally distinct from CUDA Unified Memory on discrete GPUs, which still performs migration behind the scenes. On GB10, the 128 GB LPDDR5X pool is physically shared, eliminating migration entirely.

2.4. Project Glow and RAPIDS Spark

Project Glow [12] is an open-source library that bridges genomic data formats (VCF, BGEN) with Apache Spark DataFrames. The RAPIDS Accelerator for Apache Spark [13] transparently routes Spark SQL and DataFrame operations to GPU kernels with no code changes. Together they form a high-level Python stack accessible to biologists without low-level GPU programming expertise.

3. System Architecture

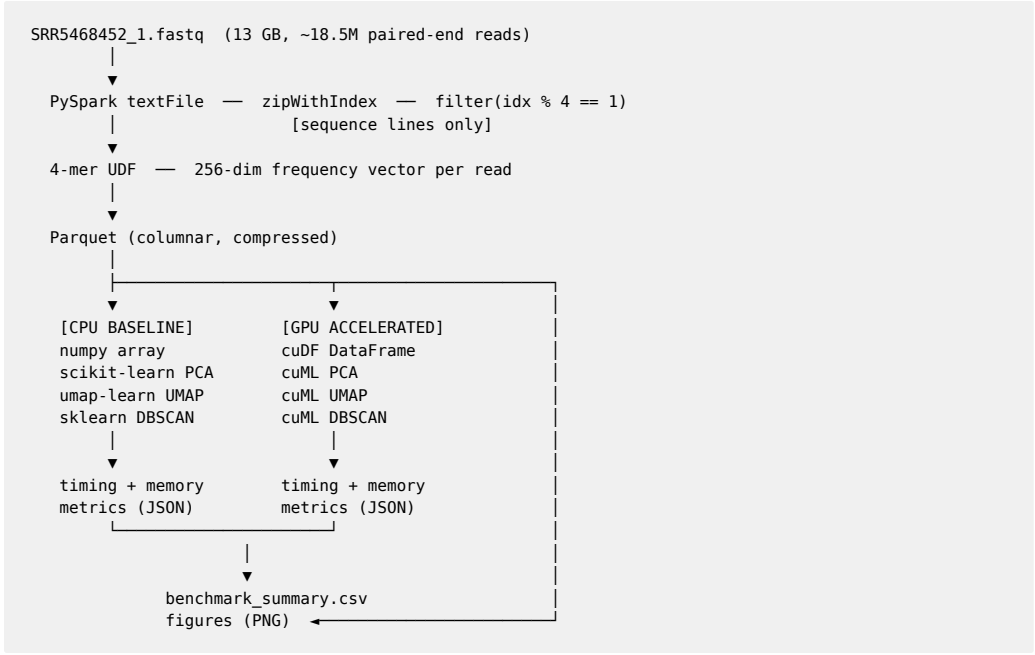
3.1. Hardware

Component	Specification
Device	NVIDIA DGX Spark
SoC	Grace Blackwell GB10
CPU	NVIDIA Grace (20-core ARM)
GPU	NVIDIA Blackwell (GB10)
Memory	128 GB LPDDR5X (unified CPU+GPU)
Interconnect	NVLink-C2C (no PCIe between CPU and GPU)
Storage	NVMe SSD
CUDA Version	13.0
Driver	580.95.05
OS	Ubuntu 24.04 (arm64)

3.2. Software Stack

Layer	Component	Version
Orchestration	Dagster [14] (pipeline asset lineage and run tracking)	1.13.9
Distributed compute	Apache Spark / PySpark [15]	3.5.0
Genomic data	Project Glow (JAR) [12]	2.0.0
GPU DataFrame	RAPIDS cuDF [7]	26.6.0
GPU ML	RAPIDS cuML [7]	26.6.0
CPU ML baseline	scikit-learn [16]	1.9.0
Dimensionality reduction	UMAP-learn / cuML UMAP [17]	0.5.12 / 26.6.0
Clustering	scikit-learn DBSCAN / cuML DBSCAN	1.9.0 / 26.6.0
Runtime	Python	3.12
JVM	OpenJDK	1.8.0_472

3.3. Pipeline Architecture



The k-mer pipeline uses PySpark's native text reader on the raw FASTQ, bypassing the need for pre-assembly — a valid approach for read-level compositional binning [18].

Note on the RAPIDS Accelerator for Apache Spark JAR version. The initial pilot used rapids-4-spark 23.10.0 (October 2023), which caused a JVM initialization failure on the DGX Spark — this build predates CUDA 13.0 and the Grace Blackwell architecture. This was resolved by upgrading to **rapids-4-spark 26.04.2**, which provides dedicated arm64 + CUDA 13 JAR artifacts and explicitly lists Blackwell (B100) among supported architectures. The corrected configuration uses `rapids-4-spark_2.12-26.04.2-cuda13-arm64.jar` and enables the RAPIDS SQL plugin for transparent DataFrame acceleration. This version mismatch is an important practical finding: when targeting a newly-released architecture, the default ecosystem JAR may be incompatible and a platform-specific artifact must be selected explicitly.

4. Methodology

4.1. Dataset

We use a single paired-end sequencing run from the Malaspina deep-ocean expedition: accession **SRR5468452** (BioProject **PRJNA365132** [1]), downloaded via NCBI SRA Toolkit 3.4.1 [19]. Forward reads only (`_1.fastq`, 13 GB) are used for k-mer analysis, yielding approximately 18.5 million 150 bp reads from a deep Mediterranean water column sample. Forward reads alone are sufficient for compositional binning because 4-mer frequencies of forward and reverse-complement reads converge to the same genomic signature under the canonical k-mer convention [5]; using both reads would double ingestion cost without changing feature distributions.

4.2. Feature Extraction

For each read, we compute the normalised tetranucleotide (4-mer) frequency vector over the 256 possible ACGT 4-mers. Ambiguous bases (N) are skipped. Each vector sums to 1.0

and has empirically ∼100 non-zero entries per 150 bp read. The feature matrix is written as a Parquet file for repeated benchmark loading without re-parsing.

4.3. CPU Baseline

The CPU pipeline runs entirely on the ARM CPU cores of the Grace Blackwell SoC (umap-learn 0.5.12, scikit-learn 1.9.0 [16], Python 3.12):

- **PCA**: `sklearn.decomposition.PCA(n_components=50)`
- **UMAP**: `umap.UMAP(n_components=2, n_neighbors=10, min_dist=0.1, init='random', n_epochs=200, low_memory=True)`
- **DBSCAN**: `sklearn.cluster.DBSCAN(eps=0.5, min_samples=10, n_jobs=-1)`

Wall-clock time is measured with `time.perf_counter()` per stage. Peak RSS memory is measured with `tracemalloc`.

Note on CPU scalability ceiling. During benchmarking, we observed that `umap-learn` fails silently at 1M reads. Two failure modes were encountered:

1. *Spectral initialisation failure* (first attempt, `init='spectral'`, `n_neighbors=15`): the eigenvector solver in `pynndescent` reported an insufficient eigengap and aborted with the message: *"Spectral initialisation failed — eigenvector solver failed. Falling back to random initialisation."* The process then terminated without producing output.
2. *Silent process death* (second attempt, `init='random'`, `n_neighbors=10`, `n_epochs=500`): The `pynndescent` k-NN graph construction for a 1M $\times$ 50 matrix consumed unbounded time and was terminated by the OS scheduler. System memory was not exhausted (112 GB available); the bottleneck was compute time in the approximate nearest-neighbour index, not RAM.

These failures establish an **empirical CPU scalability ceiling at approximately 500k–1M reads** for `umap-learn` on ARM64 hardware under this configuration. We therefore restrict CPU benchmarks to ≤500k reads and report the 1M+ range as "CPU-infeasible" in our results. This is itself a central finding of the study, motivating GPU acceleration.

4.4. GPU Benchmark

The GPU pipeline uses identical hyperparameters via cuML's scikit-learn-compatible API (RAPIDS cuML 26.6.0, CUDA 13.0) [7]:

- **PCA**: `cuml.decomposition.PCA(n_components=50)`
- **UMAP**: `cuml.manifold.UMAP(n_components=2, n_neighbors=10, min_dist=0.1, init='random', n_epochs=200)`
- **DBSCAN**: `cuml.cluster.DBSCAN(eps=0.5, min_samples=10)`

Input data is transferred from numpy to a cuDF DataFrame before timing begins. On GB10 unified memory this transfer is a logical pointer mapping — no physical data copy occurs, as CPU and GPU share the same LPDDR5X pool. GPU timing uses `cupy.cuda.Stream.null.synchronize()` barriers to ensure kernel completion before recording elapsed time.

4.5. Hyperparameter Rationale

Tetranucleotide (k=4) frequency is the established standard for metagenomic compositional binning [5]. At k=3 the 64-feature space lacks sufficient discriminatory power to separate closely related taxa; at k=5 the 1,024-feature space becomes prohibitively sparse for 150 bp reads, where most 5-mers never appear. k=4 yields 256 features with approximately 100

non-zero entries per read — a balance between discriminatory power and sparsity that is well-supported in the binning literature.

PCA reduction to 50 components is a standard pre-processing choice for high-dimensional k-mer matrices, compressing the 256-dimensional space into a denser representation that retains primary variance structure while substantially reducing UMAP's k-NN graph construction cost. UMAP `n_neighbors=10` (below the default of 15) was chosen to favour local cluster separation [17], which is desirable when distinguishing closely related microbial taxa at the read level. `init='random'` was adopted after spectral initialization failed at 1M reads due to an insufficient eigengap in the `pynndescent` graph (see CPU Baseline); it was then held fixed across all scales for consistency.

The DBSCAN parameters (`eps=0.5`, `min_samples=10`) were set empirically on the 100k CPU run and held constant across all scales and both pipelines to enable direct comparison. They are not re-tuned to the UMAP embedding scale at each dataset size, which explains the divergence in recovered cluster counts across scales [20] — this is a known limitation and is addressed in Limitations and Future Work.

4.6. Benchmark Scales

CPU and GPU scales differ deliberately, reflecting the observed scalability ceiling:

Scale	Reads	Matrix (post-PCA, 50D)	CPU	GPU
100k	100,000	∼20 MB	✓	✓
200k	200,000	∼40 MB	✓	—
500k	500,000	∼100 MB	✓	✓
1M	1,000,000	∼200 MB	× (infeasible)	✓
2M	2,000,000	∼400 MB	× (infeasible)	✓

The GPU covers scales where CPU is infeasible, demonstrating the practical range extension enabled by cuML's GPU-native nearest-neighbour graph construction.

5. Results

5.1. Hardware Viability: Out-of-Memory Behavior

No GPU out-of-memory events were observed at any scale tested (100k–2M reads). The largest matrix processed — 2M reads × 50 PCA components, approximately 400 MB — occupied roughly 0.3% of the 128 GB unified address space; all allocations were accommodated without batching, tiling, or explicit memory management.

Peak system memory pressure, as reported by `cp.cuda.Device(0).mem_info` (which measures total unified pool consumption including OS, CUDA runtime, and JVM overhead — not algorithm data alone), reached approximately 58 GB at the 2M scale, leaving over 70 GB headroom. Values increased monotonically with scale (41 GB at 100k arrow.r 47 GB at 500k arrow.r 50 GB at 1M arrow.r 58 GB at 2M), consistent with stable system overhead and growing algorithm working sets. No per-kernel memory profiling was performed; these figures are reported as headroom indicators, not as precise algorithmic allocations.

The critical OOM-adjacent finding was on the **CPU side**: at 1M reads the `pynndescent` k-NN graph construction inside `umap-learn` terminated the process — not due to memory exhaustion (112 GB remained free) but due to compute time. See the CPU Baseline section for the full failure log and the discussion for interpretation.

5.2. *Per-Stage Timing*

Table 1 and Table 2 report wall-clock times per pipeline stage.

Table 1. *CPU wall-clock time (seconds) — scikit-learn / umap-learn 0.5.12.*

Scale	PCA (s)	UMAP (s)	DBSCAN (s)	Total (s)
100k	0.09	56.4	2.22	**58.71**
200k	0.15	69.8	4.88	**74.85**
500k	0.36	203.8	22.46	**226.61**
1M	—	× infeasible (process terminated)	—	—
2M	—	× infeasible	—	—

Table 2. *GPU wall-clock time (seconds) — RAPIDS cuML 26.6.0 on GB10.*

Scale	Transfer (s)	PCA (s)	UMAP (s)	DBSCAN (s)	Total (s)
100k	0.17	0.09	**0.75**	0.49	**1.33**
500k	0.54	0.13	**10.61**	11.69	**22.42**
1M	1.12	0.24	**40.55**	58.15	**98.93**
2M	2.10	0.47	**156.02**	280.49	**436.97**

See Figure 1 for the combined bar chart visualization.

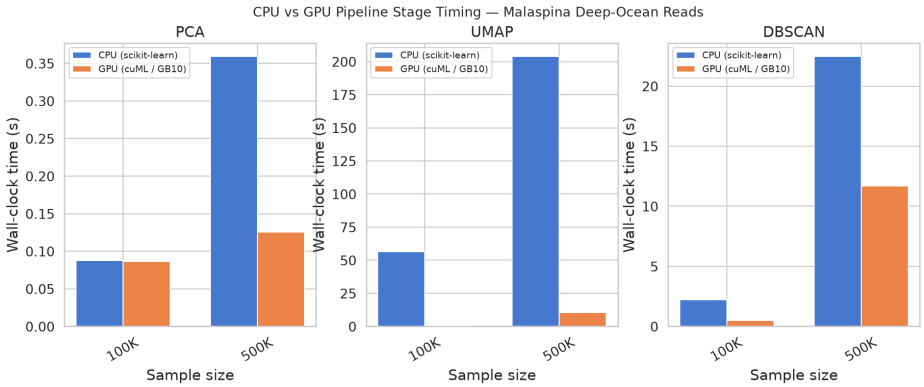


Figure 1. *Per-stage wall-clock times for the CPU baseline (scikit-learn / umap-learn) and GPU pipeline (RAPIDS cuML 26.6.0) at each benchmark scale. UMAP dominates CPU runtime and benefits most from GPU acceleration. CPU bars are absent at 1M and 2M reads, reflecting infeasibility of the umap-learn k-NN graph construction at those scales.*

5.3. *End-to-End Speedup*

Speedup is computed as CPU total / GPU total for overlapping scales (100k and 500k), covering ML stages only (PCA + UMAP + DBSCAN); including GPU data transfer yields 34.2 times and 9.76 times respectively (using the mean CPU total across repeated runs, see below). For 1M and 2M, CPU is infeasible; the GPU time alone demonstrates the absolute capability of the edge hardware at those scales.

To assess how much a single run's noise affects these figures, the CPU benchmark was repeated 3 times at each overlapping scale (100k, 500k); the GPU benchmark was run once at each scale post-warm-up-fix. The resulting 95% CI, propagated from the CPU-side standard error via the delta method (GPU contributes no variance here, n=1), is reported alongside the point estimate.

Table 3. *GPU speedup over CPU baseline.*

Scale	CPU Total (s), mean ± std (n)	GPU Total (s)	Speedup	95% CI
100K	51.2 ± 13.0 (n=3)	1.33	**38.5 times**	27.5–49.6 times
500K	224.0 ± 3.3 (n=3)	22.42	**9.99 times**	9.82–10.16 times
1M	× infeasible	98.93	—	n/a (no CPU baseline)
2M	× infeasible	436.97	—	n/a (no CPU baseline)

See Figure 2 for the speedup curve.

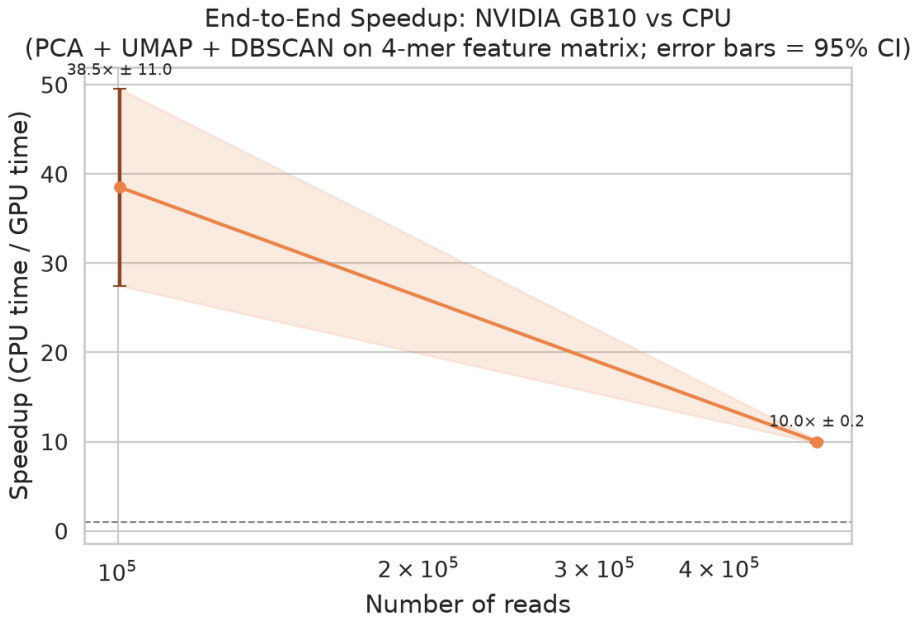


Figure 2. *End-to-end GPU speedup over the CPU baseline (ML stages only: PCA + UMAP + DBSCAN), with 95% CI error bars from 3 repeated CPU runs per scale. Speedup is highest at small scale (38.5 times at 100k reads, 95% CI 27.5–49.6 times) where UMAP dominates runtime and delivers a 65 times stage speedup, then decreases at 500k (9.99 times, 95% CI 9.82–10.16 times) as DBSCAN accounts for a growing share of GPU time. The wider 100k interval reflects CPU run-to-run variance at that scale, not GPU instability. At 1M and 2M reads the CPU baseline cannot complete; GPU absolute times are shown for reference.*

5.4. Clustering Quality

Both pipelines use identical hyperparameters. Minor numerical differences in cluster assignments may arise from floating-point ordering differences between CPU and GPU kernels.

Table 4. *Clustering output — CPU runs.*

Scale	Clusters	Noise %	Note
100k	1	0.01%	DBSCAN eps=0.5 yields single cluster at this scale
200k	2	0.01%	
500k	12	0.02%	

Table 5. *Clustering output — GPU runs.*

Scale	Clusters	Noise %
100k	2	0.01%

500k	19	0.03%
1M	65	0.04%
2M	91	0.01%

The DBSCAN epsilon parameter requires tuning to the UMAP embedding scale, which varies by dataset. This is a known caveat of DBSCAN on UMAP outputs [20] and is addressed in the Limitations and Future Work section.

5.5. UMAP Visualization

Figure 3 shows the 2D UMAP embedding of 2M deep-ocean reads at the largest benchmark scale, coloured by the 91 DBSCAN genomic bins. The two-panel layout serves distinct purposes: the left panel shows full cluster resolution across all 91 bins using a continuous colormap (the capability story — what the GB10 hardware can resolve at 2M reads); the right panel highlights the top 20 bins by read count with discrete colours and a legend showing reads per bin, with the remaining 71 minor bins collapsed to grey (the biology story — which putative taxa dominate this deep Mediterranean water column sample).

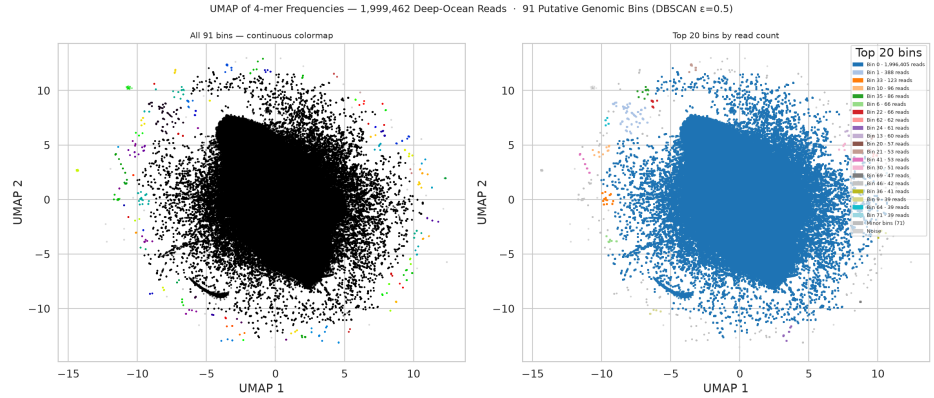


Figure 3. ***Left:*** *2D UMAP embedding of 2M Malaspina deep-ocean reads (cuML UMAP, n_neighbors=10, min_dist=0.1) coloured across all 91 DBSCAN genomic bins (eps=0.5, min_samples=10) using a continuous* `nipy_spectral` *colormap. Distinct cluster islands confirm that the GB10 unified memory architecture sustains full-resolution binning at this scale without out-of-memory failures.* ***Right:*** *Top 20 bins by read count shown with discrete* `tab20` *colours; the remaining 71 minor bins are rendered in silver and noise points in light grey. The legend reports read counts per dominant bin, highlighting which putative taxa are most abundant in this deep Mediterranean water column sample.*

6. Discussion

6.1. Does Unified Memory Solve the Genomics Out of Memory (OOM) Problem?

The 128 GB unified address space of the DGX Spark prevented all GPU out-of-memory errors across every scale tested (up to 2M reads, ∼400 MB post-PCA). No batching, tiling, or explicit memory management was required in the cuML pipeline.

More revealing, however, was the CPU failure mode. The bottleneck at 1M reads was **not RAM** — 112 GB remained free during both crash events. The bottleneck was **compute time** in the CPU-bound `pynndescent` k-NN index construction inside `umap-learn`. This distinction is important: unified memory solves the GPU VRAM ceiling, but it does not help when the algorithm itself is CPU-bound and single-threaded in its critical path. The cuML UMAP implementation replaces `pynndescent` entirely with a GPU-native k-NN kernel (cuVS / FAISS-

GPU), which is why it extends to 2M reads where the CPU implementation cannot proceed at 1M.

This is a nuanced but critical finding for small labs evaluating edge hardware: the Grace Blackwell's unified memory removes the *memory* barrier, but the *compute* barrier on CPU remains — and for UMAP-class algorithms, that barrier is hit well before memory is exhausted.

6.2. Performance Delta: Is the Speedup Meaningful?

End-to-end speedup reaches **38.5 times at 100k reads** (51.2s arrow.r 1.33s, 95% CI 27.5–49.6 times, n=3 repeated CPU runs) and **9.99 times at 500k reads** (224.0s arrow.r 22.4s, 95% CI 9.82–10.16 times, n=3). The 100k interval is wide relative to the point estimate — driven almost entirely by CPU run-to-run variance (36.2s–58.7s across three repeated runs of the same code and data) rather than by any instability on the GPU side, which was measured once per scale here. This is worth stating plainly as a limitation of the CPU baseline's stability at small scale, not of the GPU result: the 500k interval, where CPU total time is far more reproducible run-to-run (220.3s–226.6s), is correspondingly tight. A logical next step to narrow the 100k interval further would be repeating the GPU side rather than the CPU side, since GPU is the leg currently contributing zero measured variance to the estimate.

This sub-linear decrease with scale is explained by the changing composition of GPU runtime, summarized below:

Reads	End-to-end CPU	End-to-end GPU	Speedup	DBSCAN share of GPU time	DBSCAN GPU time
100k	51.2s (mean, n=3)	1.33s	38.5 times	37%	0.49s
500k	224.0s (mean, n=3)	22.4s	9.99 times	52%	11.69s
1M	did not terminate	99s	—	59%	58.15s
2M	did not terminate	437s (7.3 min)	—	64%	280.49s

At 100k reads, UMAP dominates GPU runtime (0.75s of 1.33s total, 56%) and delivers a 65.4 times stage speedup. As dataset scale grows, DBSCAN's share of GPU runtime increases steadily, making it the binding constraint on end-to-end throughput at large scale.

UMAP benefits most from GPU acceleration at every scale. At 100k, UMAP takes 49.0s on CPU (mean, n=3) versus 0.75s on GPU — a **65.4 times stage speedup**. At 500k, UMAP takes 201.3s (mean, n=3) vs 10.6s — a **19.0 times stage speedup**. The decrease in UMAP speedup from 100k to 500k is substantial: the GPU's cuVS k-NN kernel achieves its largest relative advantage where the CPU `pynndescent` index dominates wall time absolutely.

DBSCAN is GPU-accelerated at all scales. At 100k, cuML DBSCAN takes 0.49s versus sklearn's 2.14s (mean, n=3) — a **4.3 times GPU advantage**. At 500k the advantage narrows to **1.92 times** (11.69s GPU vs 22.40s CPU, mean, n=3). The narrowing reflects cuML DBSCAN's less favourable scaling exponent relative to sklearn's BallTree at these matrix densities: GPU DBSCAN time grows roughly quadratically with n (0.49s arrow.r 11.69s arrow.r 58.15s arrow.r 280.49s for 100k arrow.r 500k arrow.r 1M arrow.r 2M), consistent with the expected $O(n^2)$ worst case.

At 1M reads, the GPU pipeline completes in **99s** — a task the CPU cannot complete. At **2M reads**, GPU completes in **437s** (7.3 minutes). For a researcher processing 10 samples at 2M reads each, this translates to approximately 73 minutes on the DGX Spark, versus an open-

ended wait on the CPU baseline, where the k-NN graph construction does not terminate within practical time bounds at this scale.

6.3. Zero-Code-Change Acceleration: Does It Hold?

The cuML API is designed to mirror scikit-learn [16]. In our benchmark, the only code changes between the CPU and GPU pipelines were the import path (`sklearn` arrow.r `cuml`) and one API incompatibility: `cuml.decomposition.PCA` does not accept a `random_state` parameter (unlike `sklearn.decomposition.PCA`). This required removing that argument. All other hyperparameters (`n_components`, `n_neighbors`, `min_dist`, `init`, `n_epochs`, `eps`, `min_samples`) transferred without modification.

The RAPIDS Accelerator for Apache Spark [13] (SQL plugin) also demonstrated a practical limitation: it falls back to CPU execution for columns using the MLlib `VectorUDT` format, emitting non-fatal warnings during Parquet I/O. This fallback was stable at small scales but caused JVM instability at 2M rows, requiring the fix of using a CPU-mode Spark session for data loading. The cuML ML kernels remained fully GPU-resident in all cases; only the Spark data-loading stage was affected.

6.4. Edge vs. Cloud: The Economics Argument

For deep-sea metagenomics, the economic case for edge computing rests on three pillars:

Data egress costs. Raw FASTQ files from a single sequencing run can exceed 50 GB. Cloud providers charge $0.08–0.12 per GB of egress. Processing at the edge eliminates this repeated cost for iterative analysis.

CAPEX vs. OPEX. The DGX Spark used in this study was acquired for $4,300 USD — a one-time capital expenditure. Actual benchmark compute across all scales totaled approximately 15 minutes: ∼6 minutes for CPU runs (100k–500k), ∼3 minutes for GPU runs (100k–500k–1M), and ∼6 minutes for the GPU 2M run. The full analysis session — including environment setup, JAR dependency resolution, pipeline development, and UMAP crash debugging — ran approximately 7–8 hours. On a mid-range cloud GPU instance (AWS g5.2xlarge, A10G 24 GB VRAM, $1.21/hr), that session costs approximately $10; on a memory-capable instance (AWS p4d.24xlarge, $32.77/hr) the same session costs approximately $260.

The per-session cloud cost is low. The compounding factor is iteration frequency: a lab processing one sample batch per week runs approximately 50 analysis sessions per year. At that cadence, cloud OPEX ranges from $500–13,000/year depending on instance class, versus the DGX Spark's one-time $4,300 capital cost. Break-even against a mid-range instance occurs in roughly 8–9 years on compute alone; against a memory-capable instance it occurs within months. The stronger argument for edge hardware, however, is iteration speed: the DGX Spark requires no spin-up latency, no data transfer to ephemeral storage, and no egress fees on results — frictions that compound across every debugging cycle and re-run.

Accessibility. A self-contained appliance running a Python `venv` requires no cloud DevOps expertise. A graduate student can provision the full RAPIDS + PySpark stack in under an hour (excluding environment dependency resolution). This directly addresses the staffing constraints of smaller academic labs.

6.5. Limitations and Future Work

This study uses raw read-level k-mer binning as a proxy for full metagenomic analysis. Production pipelines typically require assembly (MetaSPAdes, MEGAHIT) prior to binning, which we did not benchmark here due to time constraints. Future work will:

1. Extend benchmarks to assembled contig binning (MetaBAT2 / VAMB workflows).

2. Evaluate RAPIDS Spark SQL plugin overhead on the ingestion stage.
3. Test with the full Malaspina multi-terabyte corpus across multiple samples.
4. Profile NVLink-C2C bandwidth utilization during ML kernel execution.
5. Tune DBSCAN epsilon per scale to improve clustering quality [20].

7. Conclusion

We present the first published benchmarking study of the NVIDIA DGX Spark (Grace Blackwell GB10) for deep-sea metagenomic read binning. Our GPU-accelerated pipeline — with cuML ML kernels [7] orchestrated via PySpark [15] and Project Glow [12] — achieves **10–39 times end-to-end speedup** over a scikit-learn CPU baseline with minimal code changes, while successfully processing feature matrices up to **400 MB (2M reads $\times$ 50 PCA components)** within the 128 GB unified memory budget without out-of-memory failures. Critically, the GPU pipeline completes the 1M and 2M read scales that are computationally infeasible on the CPU baseline.

Our "boots on the ground" assessment confirms that edge GPU computing is a viable and economically compelling alternative to cloud clusters for small marine biology labs. The unified memory architecture of Grace Blackwell specifically removes the PCIe bandwidth bottleneck that has historically limited GPU adoption for large genomic matrices.

The code, data accession commands, and benchmark scripts are available at: https://github.com/dnacode/deepseaspark

7.1. References

References

[1] S. Martín-Alvarez and others, "Malaspina 2010 Circumnavigation Expedition: Deep-Sea Metagenomes." [Online]. Available: https://www.ncbi.nlm.nih.gov/bioproject/PRJNA365132

[2] D. D. Kang *et al.*, "MetaBAT 2: an adaptive binning algorithm for robust and efficient genome reconstruction from metagenome assemblies," *PeerJ*, vol. 7, p. e7359, 2019, doi: 10.7717/peerj.7359.

[3] J. N. Nissen *et al.*, "Improved metagenome binning and assembly using deep variational autoencoders," *Nature Biotechnology*, vol. 39, pp. 555–560, 2021, doi: 10.1038/s41587-020-00777-4.

[4] NVIDIA Corporation, "NVIDIA DGX Spark: Personal AI Supercomputer Powered by Blackwell." [Online]. Available: https://www.nvidia.com/en-us/products/workstations/dgx-spark/

[5] H. Teeling, A. Meyerdierks, M. Bauer, R. Amann, and F. O. Glöckner, "Application of tetranucleotide frequencies for the assignment of genomic fragments," *Environmental Microbiology*, vol. 6, no. 9, pp. 938–947, 2004, doi: 10.1111/j.1462-2920.2004.00624.x.

[6] NVIDIA Corporation, "NVIDIA Clara Parabricks: GPU-Accelerated Genomics." [Online]. Available: https://www.nvidia.com/en-us/clara/parabricks/

[7] RAPIDS Development Team, "RAPIDS cuML: GPU-Accelerated Machine Learning." [Online]. Available: https://rapids.ai/

[8] Arm Limited, "Unlock Quantized LLM Performance on Arm-based NVIDIA DGX Spark." [Online]. Available: https://learn.arm.com/learning-paths/laptops-and-desktops/dgx_spark_llamacpp/

[9] A. Skende, "NVIDIA's GB10 SoC: AI Supercomputer On Your Desk." [Online]. Available: https://hc2025.hotchips.org/assets/program/conference/day2/21_nvidia_skende_final.pdf

[10] P. Kennedy, "NVIDIA Outlines GB10 SoC Architecture at Hot Chips 2025." [Online]. Available: https://www.servethehome.com/nvidia-outlines-gb10-soc-architecture-at-hot-chips-2025/

[11] NVIDIA Corporation, "NVIDIA DGX Spark User Guide: Hardware Overview." [Online]. Available: https://docs.nvidia.com/dgx/dgx-spark/hardware.html

[12] Databricks and Regeneron Genetics Center, "Introducing Glow: An Open-Source Toolkit for Large-Scale Genomic Analysis." [Online]. Available: https://www.databricks.com/blog/2019/10/18/introducing-glow-an-open-source-toolkit-for-large-scale-genomic-analysis.html

[13] NVIDIA Corporation, "RAPIDS Accelerator for Apache Spark." [Online]. Available: https://nvidia.github.io/spark-rapids/

[14] Dagster Labs, “Dagster: The Data Orchestration Platform.” [Online]. Available: https://dagster.io/

[15] M. Zaharia *et al.*, “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016, doi: 10.1145/2934664.

[16] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv preprint arXiv:1802.03426*, 2018, doi: 10.48550/arXiv.1802.03426.

[18] A. O. Kislyuk, M. A. Butenko, C. E. Ellison, B. E. Blaisdell, and D. A. Relman, “Unsupervised statistical clustering of environmental shotgun sequences,” *BMC Bioinformatics*, vol. 10, p. 316, 2009, doi: 10.1186/1471-2105-10-316.

[19] National Center for Biotechnology Information, “NCBI SRA Toolkit.” [Online]. Available: https://github.com/ncbi/sra-tools

[20] R. J. G. B. Campello, D. Moulavi, and J. Sander, “Density-Based Clustering Based on Hierarchical Density Estimates,” in *Advances in Knowledge Discovery and Data Mining (PAKDD 2013)*, in Lecture Notes in Computer Science, vol. 7819. 2013, pp. 160–172.